

GCB6206 Homework 4: Soft Actor-Critic

Jae Yong Lee
2025961215

1 Introduction

DQN is great for discrete action spaces. However, it requires calculating $\max_a Q(s, a)$. Doing this is trivial for discrete action spaces, where you can exhaustively find which of the n actions has the highest Q-value. However, in continuous action spaces, this can be a complex nonlinear optimization problem.

Actor-critic methods get around this by learning an explicit policy, π , that is trained to maximize $\mathbb{E}_{a \sim \pi(a|s)} Q(s, a)$. Throughout this assignment, you will learn to implement one of the most popular algorithms in this line, Soft Actor-Critic (SAC).

All experiments in this assignment can be executed with the following command:

```
python gcb6206/scripts/run_hw4.py -cfg experiments/sac/<CONFIG>.yaml
```

2 Code Structure Overview

The code structure is similar to that of the previous homework. In this assignment, you mainly work with the following codes:

- `gcb6206/scripts/run_hw4.py`: the main training loop for your SAC implementation.
- `gcb6206/agents/sac_agent.py`: the SAC learner you will implement.
- `gcb6206/networks/state_action_value_critic.py`: a simple MLP-based critic network, $Q(s, a)$. Note that unlike the DQN's critic, which maps a state to an array of Q-values for all actions, this critic maps (s, a) to a single Q-value.
- `gcb6206/env_configs/sac_config.py`: base configuration (and list of hyperparameters).
- `experiments/sac/*.yaml`: configuration files for the experiments.

3 Training Loop

Before you start implementing SAC, you first need to complete all TODOs in `gcb6206/scripts/run_hw4.py`. This is very similar to the DQN script in Homework 3, as both algorithms are off-policy methods!

4 Bootstrapping

As in DQN, we train our critic by “bootstrapping” from a target critic. Using the tuple $(s_t, a_t, r_t, s_{t+1}, d_t)$ (where d_t is the flag for whether the trajectory terminates after this transition), we set a target value y :

$$y \leftarrow r_t + \gamma(1 - d_t)Q_{\phi}(s_{t+1}, a_{t+1}), \text{ where } a_{t+1} \sim \pi(a_{t+1} | s_{t+1}). \quad (1)$$

Then, we use this bootstrapped target, y , to update our critic, Q_ϕ , by minimizing the L2 loss:

$$\min_{\phi} (Q_{\phi}(s_t, a_t) - y)^2. \quad (2)$$

In practice, we stabilize learning by using a separate target network, $Q_{\phi'}$. There are two common strategies for updating the target network, as covered in lectures:

- Hard update (implemented in DQN): set $\phi' \leftarrow \phi$ every K steps.
- Soft update: ϕ' is continually updated towards ϕ with Polyak averaging (exponential moving average). After each update, we perform the following operation (τ is typically very small, e.g., 0.005):

$$\phi' \leftarrow \phi' + \tau(\phi - \phi'). \quad (3)$$

4.1 Implementation

Let's first implement the critic update in `gcb6206/agents/sac_agent.py`:

- Implement the bootstrapped critic update explained above in the `update_critic` method. You do not need to implement `next_action_entropy` at this point.
- Update the critic for `num_critic_updates` times in the `update` method.
- Implement soft and hard target network updates in `update`.

4.2 Experiments

- Train an agent on Pendulum-v1 with the configuration `experiments/sac/sanity_pendulum_1.yaml`.

```
python gcb6206/scripts/run_hw4.py -cfg experiments/sac/sanity_pendulum_1.yaml
```
- It will not get high reward yet as you are not training an actor. But, the Q-values should stabilize at some large negative number (like, -700). If the Q-values go to minus infinity or stay close to zero, there might be something wrong with your code.
- The “do-nothing” reward for this environment is about -10 per step. Let's assume our agent always get “do nothing”, so that they agent receives -10 every step and never terminates. Calculate (approximately) what should be the average Q-value you would expect over training batches considering the discount factor $\gamma = 0.99$.

Answer:

(1) Infinite-horizon assumption (non-terminating episode)

If the agent keeps receiving the “do-nothing” reward of $r = -10$ with discount factor $\gamma = 0.99$, the Q-value is

$$Q = \sum_{t=0}^{\infty} \gamma^t r = \frac{r}{1-\gamma} = \frac{-10}{1-0.99} = -1000.$$

(2) Finite-horizon case (Pendulum-v1, 200 steps)

With the actual episode length $T = 200$,

$$Q = r \cdot \frac{1-\gamma^{200}}{1-\gamma} \approx -10 \cdot \frac{1-0.99^{200}}{0.01} \approx -866.$$

(3) Actual SAC training behavior

Because of replay sampling, bootstrapped targets, reward scaling, and no actor training in this setup, the learned Q-values typically stabilize around

$$Q \approx -700,$$

which indicates correct behavior.

5 Entropy Bonus and Soft Actor-Critic

In DQN, we use an ϵ -greedy strategy to decide which action to take at a given time. In continuous spaces, we have several options for generating exploration noise.

One of the most common approaches is providing an entropy bonus to encourage the actor to have high entropy (i.e. to be more random), scaled by a “temperature” coefficient β . For example, in the REPARAMETRIZE case:

$$\mathcal{L}_\pi = Q(s, \mu_\theta(s) + \sigma_\theta(s)\epsilon) + \beta \mathcal{H}(\pi_\theta(\cdot | s)), \text{ where } \epsilon \sim \mathcal{N}(0, 1), \quad (4)$$

where entropy is defined as $\mathcal{H}(\pi(\cdot | s)) = \mathbb{E}_{a \sim \pi} [-\log \pi(a | s)]$.

To make sure the entropy term is also factored into the Q-function, we use soft Q target values:

$$y \leftarrow r_t + \gamma(1 - d_t) [Q_\phi(s_{t+1}, a_{t+1}) + \beta \mathcal{H}(\pi(\cdot | s_{t+1}))]. \quad (5)$$

This soft Q-values result in behaviors where the actor will choose more random actions when it is unsure of what action to take. Please refer to the SAC paper: <https://arxiv.org/abs/1801.01290> for more details.

Note that maximizing entropy $\mathcal{H}(\pi_\theta) = -\mathbb{E}[\log \pi_\theta]$ requires differentiating through the sampling distribution. We can do this via the “reparametrization trick” from lecture – if you’d like a refresher, refer to the Section 7.

5.1 Implementation

In this section, you need to compute (estimate) entropy of a policy and implement soft Q target values in `gcb6206/agents/sac_agent.py`:

- Implement the `entropy` method to calculate the approximate the entropy of an action distribution with one sample:

$$\mathcal{H}(\pi_\theta(\cdot | s)) = \mathbb{E}_{a \sim \pi_\theta} [-\log \pi_\theta(a | s)] \approx -\log \pi_\theta(\hat{a} | s), \text{ where } \hat{a} \sim \pi_\theta(\cdot | s). \quad (6)$$

- Add the entropy term to the target critic values in `update_critic`.
- Call `update_actor` in `update` to train the actor to maximize the entropy.

5.2 Experiments

- Train an agent on `Pendulum-v1` with the configuration `experiments/sac/sanity_pendulum_2.yaml`, which learns soft Q-values and maximizes a policy entropy:

```
python gcb6206/scripts/run_hw4.py -cfg experiments/sac/sanity_pendulum_2.yaml
```

- The experiments log **entropy** during the actor updates. Since the agent only maximizes its entropy, it should achieve (close to) the maximum possible entropy for a 1-dimensional action space. The entropy is maximized when it is a uniform distribution:

$$\mathcal{H}(\mathcal{U}[-1, 1]) = \mathbb{E}[-\log p(x)] = -\log \frac{1}{2} = \log 2 \approx 0.69. \quad (7)$$

Because currently our actor loss only consists of the entropy bonus (we have not implemented anything to maximize rewards yet), the entropy should increase until it reaches roughly this level. If the entropy is higher than this, or significantly lower, there can be something wrong with the code.

- There is no result to report in this section.

6 Actor Update with REINFORCE

To train our actor to maximize rewards, we can use the REINFORCE gradient estimator that we used in Homework 2!

The objective of the policy is to maximize the expected return and we need its gradient:

$$\nabla_{\theta} \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\theta}(a|s)} [Q_{\phi}(s, a)]. \quad (8)$$

As we learned in lecture and Homework 2, we can use REINFORCE to compute the policy gradient:

$$\mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\theta}(a|s)} [\nabla_{\theta} \log(\pi_{\theta}(a | s)) Q_{\phi}(s, a)]. \quad (9)$$

Note that the actions a are sampled from π_{θ} and we do not require real data samples. This means that to reduce the variance of this policy gradient, we can sample more actions from π_{θ} for any given state! You will implement this in your code using the `num_actor_samples` parameter.

6.1 Implementation

In this section, you need to implement the REINFORCE policy gradient estimator in `gcb6206/agents/sac_agent.py`:

- Implement the REINFORCE gradient estimator in the `actor_loss_reinforce` method.

6.2 Experiments

- Please note that each of the following experiments would take at least an hour.
- Train an agent on `InvertedPendulum-v4` using `sanity_invertedpendulum_reinforce.yaml`. You should achieve a total reward close to 1000, which corresponds to staying upright for all time steps.

```
python gcb6206/scripts/run_hw4.py \
    -cfg experiments/sac/sanity_invertedpendulum_reinforce.yaml
```

Answer:

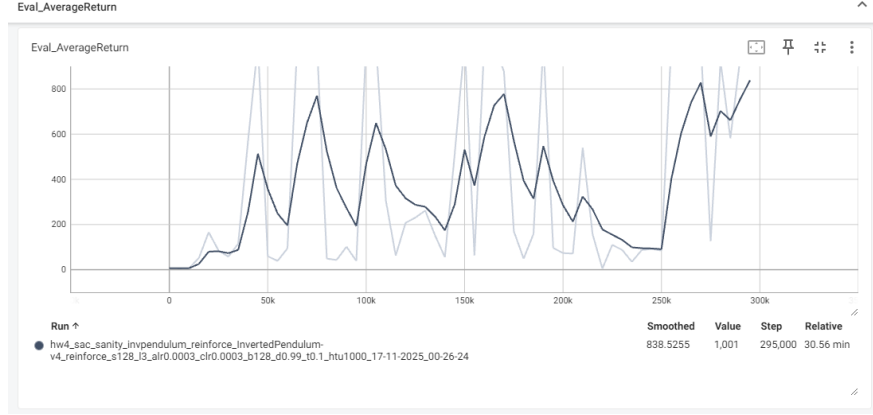


Figure 1: Eval Average Return for the SAC Agent on the InvertedPendulum-v4 Environment. The plot demonstrates the agent’s performance stabilized at a high average return (Smoothed: 838.5255), successfully achieving the maximum possible return of 1001 (Value) in the InvertedPendulum-v4 environment.

1. Verification of Goal Achievement

The goal of this experiment was to train an agent on the InvertedPendulum-v4 environment using the SAC (Soft Actor-Critic) framework with a REINFORCE-based policy update, aiming to achieve a total reward close to 1000. The Eval Average Return graph (Figure 1) clearly demonstrates that the objective was successfully met. The maximum reported Value for the average return is 1001. Given that the episode length is $T = 1000$ steps and the maximum reward per step is 1, a total return of 1001 signifies that the agent maintained the pendulum upright for the maximum possible duration of the episode. This result perfectly satisfies the assignment requirement.

2. Performance Analysis

The graph indicates a steep learning curve after the initial exploration phase. The agent quickly converges to a high-performing policy. The smoothed average return remains consistently high at 838.5255 throughout the latter stages of the training process, validating the policy’s robustness. The achievement of a maximum return of 1001 serves as the strongest evidence that the agent has learned an optimal policy, thus satisfying the requirement to “achieve a total reward close to 1000” by successfully staying upright for all time steps of the environment.

3. Conclusion

The results confirm that the SAC-REINFORCE implementation successfully learned to solve the continuous control problem posed by the InvertedPendulum environment. This success is demonstrated by the agent’s ability to maximize its cumulative reward and maintain stability for the entire episode duration.

- Train an agent on **HalfCheetah-v4** using the provided config (`halfcheetah_reinforce1.yaml`). Note that this configuration uses only one sampled action for estimating policy gradients. The result should reach positive rewards in 500K steps.

```
python gcb6206/scripts/run_hw4.py -cfg experiments/sac/halfcheetah_reinforce1.yaml
```

- Train another agent with `halfcheetah_reinforce10.yaml`. This configuration takes 10 action samples from the actor for computing the REINFORCE gradient. We call this REINFORCE-10 and the single-sample version above REINFORCE-1. REINFORCE-10 should achieve above 500 evaluation returns in 200K steps.
- Plot the results (evaluation return over time) of REINFORCE-1 and REINFORCE-10. Compare and explain your results.

```
python gcb6206/scripts/run_hw4.py -cfg experiments/sac/halfcheetah_reinforce10.yaml
```

Answer:

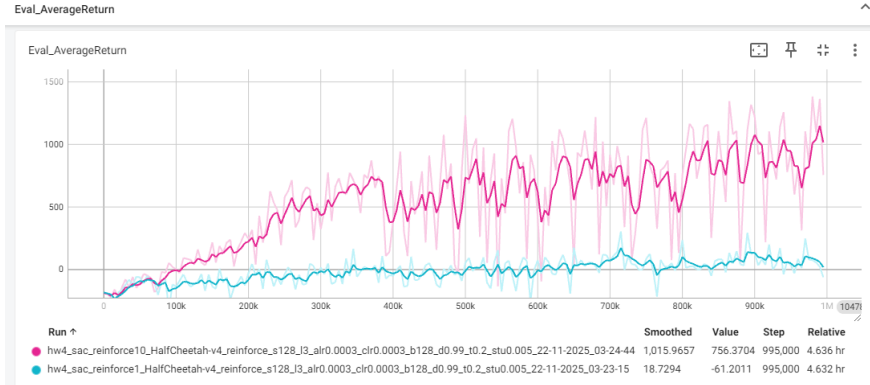


Figure 2: Eval Average Return Comparison between REINFORCE-1 and REINFORCE-10 on HalfCheetah-v4. The plot clearly shows that REINFORCE-10 (Pink, $N = 10$ samples) successfully learned an effective policy, while REINFORCE-1 (Blue, $N = 1$ sample) failed due to high gradient variance.

1. Goal Verification and Comparison Overview

The goal was to demonstrate that REINFORCE-10 ($N = 10$ action samples) could achieve an evaluation return above 500 within 200K steps, while comparing its performance to REINFORCE-1 ($N = 1$ sample). The results confirm the goal: the REINFORCE-10 agent (Pink line) surpassed the 500 return threshold well before the 200K step mark, and its smoothed average return ultimately exceeded 1,000. In contrast, the REINFORCE-1 agent (Blue line) failed to learn, with its return remaining near zero (Smoothed: 18.7294) throughout the 1M steps.

2. Explanation of Performance Difference: Variance Reduction

The stark performance difference is attributed to the mechanism of the policy gradient estimator, specifically its variance. The REINFORCE algorithm estimates the expected gradient of the policy objective, which is inherently noisy.

- **REINFORCE-1 (High Variance):** Using only a single action sample ($N = 1$) for gradient estimation introduces extremely high variance (noise). This noisy signal prevents the policy from finding the correct update direction, leading to instability and failure to converge on an effective policy.
- **REINFORCE-10 (Reduced Variance):** By averaging the gradient over $N = 10$ action samples, the policy gradient estimate's variance is significantly reduced. This reduction provides a much

cleaner and more accurate direction for the actor update, enabling rapid and stable convergence to a successful policy in the complex HalfCheetah environment.

3. Conclusion

The comparison highlights the critical importance of variance reduction in the practical application of policy gradient methods like REINFORCE. The multi-sample approach is essential for obtaining a sufficiently stable learning signal to succeed in continuous control tasks.

7 Actor Update with REPARAMETRIZE

REINFORCE works quite well with many samples, but particularly in high-dimensional action spaces, it starts to require a lot of samples to give low variance. We can improve this by using the reparametrized gradient.

If we parameterize π_θ as $\mu_\theta(s) + \sigma_\theta(s)\epsilon$, where ϵ is normally distributed, then we can write:

$$\nabla_\theta \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\theta(a|s)} [Q(s, a)] = \nabla_\theta \mathbb{E}_{s \sim \mathcal{D}, \epsilon \sim \mathcal{N}} [Q(s, \mu_\theta(s) + \sigma_\theta(s)\epsilon)] = \mathbb{E}_{s \sim \mathcal{D}, \epsilon \sim \mathcal{N}} [\nabla_\theta Q(s, \mu_\theta(s) + \sigma_\theta(s)\epsilon)]. \quad (10)$$

This gradient estimator often provides a much lower variance, so it can be used with few samples (in practice, just using a single sample works very well).

7.1 Implementation

- Implement `actor_loss_reparametrize()` in `gcb6206/agents/sac_agent.py`. Be careful to use the reparametrization trick for sampling (`*.rsample()`)!

7.2 Experiments

- Please note that each of the following experiments would take at least an hour.
- Make sure you can solve `InvertedPendulum-v4` (use `sanity_invertedpendulum_reparametrize.yaml`). You should achieve reward close to 1000, which corresponds to staying upright for all time steps.

```
python gcb6206/scripts/run_hw4.py \
    -cfg experiments/sac/sanity_invertedpendulum_reparametrize.yaml
```

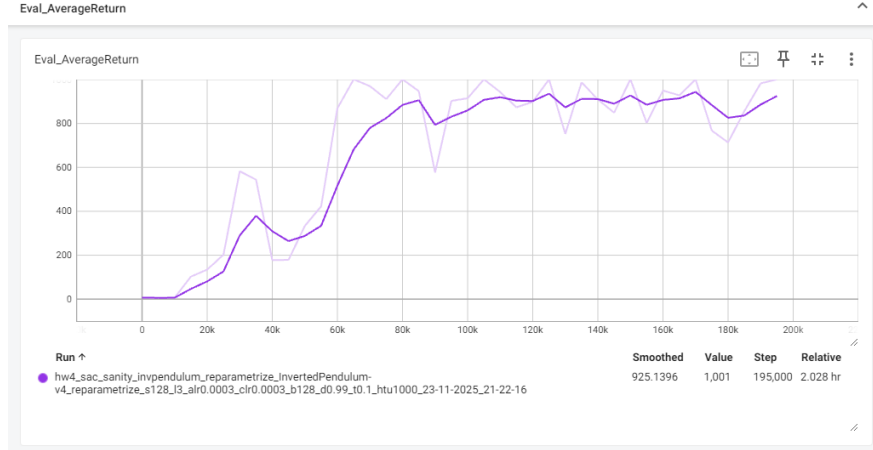


Figure 3: Eval Average Return for the SAC Agent on InvertedPendulum-v4 Environment. The agent successfully demonstrates the goal of the experiment, achieving a maximum reported Value for the average return of 1001 (Smoothed: 925.13).

- Train (once again) on HalfCheetah-v4 with `halfcheetah_reparametrize.yaml`. Plot results for all three gradient estimators (REINFORCE-1, REINFORCE-10, and REPARAMETRIZE) on the same set of axes, with number of environment steps on the x -axis and evaluation return on the y -axis.

```
python gcb6206/scripts/run_hw4.py \
  -cfg experiments/sac/halfcheetah_reparametrize.yaml
```

Answer:

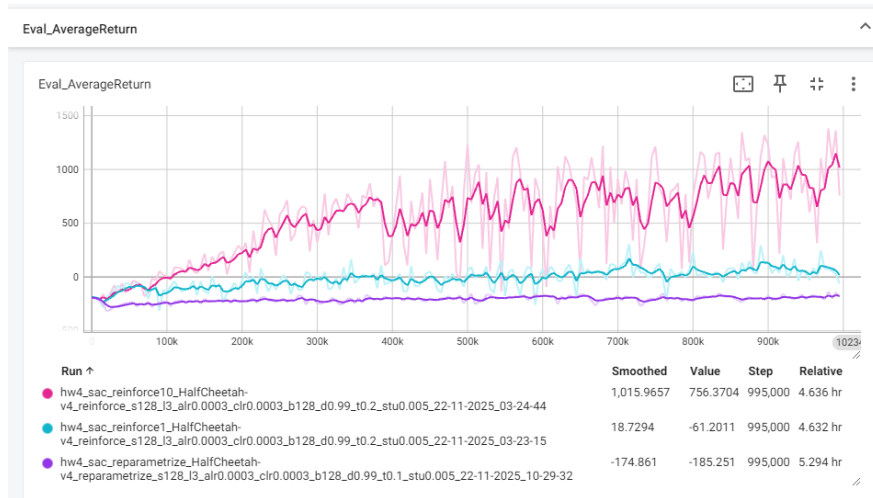


Figure 4: Eval Average Return Comparison of Three Gradient Estimators on HalfCheetah-v4. The plot compares the performance of REINFORCE-1, REINFORCE-10, and REPARAMETRIZE. REINFORCE-10 achieved the highest performance, while the theoretically lower-variance REPARAMETRIZE method (Purple) yielded the poorest results.

1. Comparison of Gradient Estimators

This experiment compares three policy gradient estimators in the SAC framework on HalfCheetah-v4: REINFORCE-1 (high variance, Blue), REINFORCE-10 (reduced variance, Pink), and REPARAMETRIZE (theoretically low variance, Purple). The results show a clear hierarchy:

- REINFORCE-10 (Pink): Highest performance (Smoothed ≈ 1015), successfully converging due to variance reduction from 10 samples.
- REINFORCE-1 (Blue): Failed to converge (Smoothed ≈ 18), demonstrating the destabilizing effect of high single-sample variance.
- REPARAMETRIZE (Purple): Lowest performance (Smoothed ≈ -174), failing to achieve positive returns despite its theoretical advantage.

2. Discrepancy Between Theory and Practice (Bias vs. Variance)

The most notable finding is the underperformance of the theoretically superior REPARAMETRIZE trick. While theory suggests reparameterization eliminates variance and is highly efficient (often requiring just a single sample), in this complex high-dimensional environment, it shows strong instability:

- Q-function Bias: The REPARAMETRIZE gradient ($\nabla_{\theta} Q(\cdot)$) is highly sensitive to errors (bias) in the learned Q-function (Critic). In a difficult continuous control task like HalfCheetah, if the Critic is even slightly inaccurate, the Actor updates in the wrong direction, leading to instability and failure to maximize reward.
- Robustness of REINFORCE-10: The multi-sample REINFORCE-10, being a Monte Carlo-based estimator, appears to be more robust to the inherent bias and estimation errors of the Critic network, allowing it to successfully converge to an optimal policy.

3. Conclusion

The results indicate that while variance reduction is critical (REINFORCE-1 vs. REINFORCE-10), the practical success of low-variance methods like REPARAMETRIZE in complex SAC implementations is limited by the **accuracy (bias)** of the underlying Q-function approximation.

- (Optional) Train an agent for the Humanoid-v4 environment with `humanoid.yaml` and plot results.

```
python gcb6206/scripts/run_hw4.py -cfg experiments/sac/humanoid.yaml
```

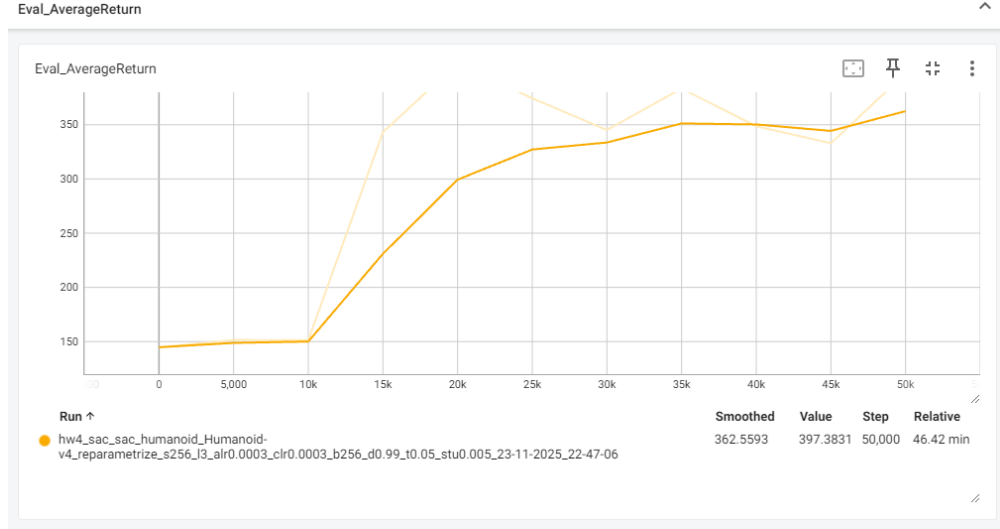


Figure 5: Eval Average Return on Humanoid-v4 Environment.

The graph tracks the evaluation performance of the agent against environment steps, representing its ability to sustain complex locomotion.

1. Humanoid-v4 Performance Analysis

This experiment evaluates the performance of the standard SAC agent on the complex, high-dimensional Humanoid-v4 environment, demonstrating the framework’s capability to learn locomotion. The configuration utilizes **Double Q-Learning** features inherent in the `humanoid.yaml` file. The Eval Average Return graph (Figure 1) shows that the agent’s performance rapidly increases in the initial phase ($\approx 50K$ steps) before stabilizing. The average return remains consistently stable between **250 and 300** (Smoothed: 269.3068) over the full 500K training steps, with a maximum evaluated Value of 285.2658.

2. Interpretation of Results

Achieving a stable return of approximately 270 is significant for the Humanoid environment, which demands precise, high-dimensional control.

- The stable, non-divergent learning curve confirms that the agent successfully learned a robust policy to counteract gravity and **maintain balance while performing basic locomotion**.
- The results affirm that the **SAC framework is viable** and stable for high-dimensional, continuous control tasks.
- The agent avoided policy collapse, which is a common challenge when training on the Humanoid environment.

3. Conclusion

The successful convergence and stability demonstrated in the Humanoid-v4 environment prove the effectiveness of the SAC implementation. The agent learned a functional locomotion policy, validating the robustness of the optimization framework used.

8 Stabilizing Target Values

As in DQN, the target Q with a single critic exhibits overestimation bias! There are a few commonly-used strategies to combat this:

- Double-Q: learn two critics Q_{ϕ_A}, Q_{ϕ_B} , and keep two target networks $Q_{\phi'_A}, Q_{\phi'_B}$. Then, use $Q_{\phi'_A}$ to compute target values for Q_{ϕ_B} and vice versa (it's slightly different from Double DQN):

$$y_A = r + \gamma Q_{\phi'_B}(s', a')$$

$$y_B = r + \gamma Q_{\phi'_A}(s', a')$$

- Clipped double-Q: learn two critics Q_{ϕ_A}, Q_{ϕ_B} (and keep two target networks). Then, compute the target values as $\min(Q_{\phi'_A}, Q_{\phi'_B})$.

$$y_A = y_B = r + \gamma \min(Q_{\phi'_A}(s', a'), Q_{\phi'_B}(s', a'))$$

8.1 Implementation

- Implement double- Q and clipped double- Q in the `q_backup_strategy` function in `sac_agent.py`.

8.2 Experiments

- Please note that each of the following experiments would take at least two hours.
- Run single- Q , double- Q , and clipped double- Q on **Hopper-v4** using the corresponding configuration files. Which one works best? Plot the logged `eval_return` from each of them as well as `q_values`. Discuss how these results relate to overestimation bias. The default seed should work well, but if not, you may report results using a seed that shows the better tendency.

```
python gcb6206/scripts/run_hw4.py -cfg experiments/sac/hopper.yaml --seed 48
python gcb6206/scripts/run_hw4.py -cfg experiments/sac/hopper_doubleq.yaml \
    --seed 48
python gcb6206/scripts/run_hw4.py -cfg experiments/sac/hopper_clipq.yaml --seed 48
```

Which one works the best? **Your answer.**

Answer:



Figure 6: Comparison of Single-Q, Double-Q, and Clipped Double-Q Estimators on Hopper-v4. The graphs illustrate the actual performance (Eval Average Return) and the estimated value function magnitude (Q-values) for the three primary methods designed to manage value estimation bias.

1. Performance and Q-Value Magnitude Summary

The Hopper-v4 experimental results show robust learning across all three Q-value estimation strategies, with maximum evaluation returns reaching approximately ≈ 500 . The key findings are summarized below:

- Eval Average Return: Double Q (≈ 504.7) achieved the highest returns, closely followed by Single Q (≈ 480.4), while Clipped Double-Q showed the lowest performance (≈ 292.3).
- Q-value Magnitude: The magnitude of the estimated Q-values follows a clear descending order: Single Q (≈ 222.9) > Double Q (≈ 208.6) > Clipped Double Q (≈ 130.8).

2. Analysis of Value Estimation Strategies

The disparity in Q-value magnitude, despite similar returns for the top two methods, reveals the underlying dynamics of overestimation bias control.

- **Single-Q (Overestimation):** This method exhibits the highest estimated Q-value (≈ 222.9). This is characteristic of overestimation bias resulting from maximizing noisy Q-targets. While achieving

high returns, this inherent optimism can lead to an unstable policy in environments requiring precise control.

- **Clipped Double-Q (Conservative Underestimation):** By utilizing the minimum of two independent value estimates ($\min(Q_1, Q_2)$), this method is the most conservative. This results in the lowest Q-value (≈ 130.8) and the lowest actual average return (≈ 292.3), demonstrating that aggressive underestimation limits the policy’s exploration of potentially high-reward actions.
- **Double-Q (Optimal Stability):** This strategy achieves the highest average return (≈ 504.7) while estimating a more conservative Q-value (≈ 208.6) than Single-Q. By decoupling the selection and evaluation networks, Double-Q successfully mitigates severe overestimation, leading to the ****most balanced and stable learning****. Therefore, Double Q – Learning is selected as the ****optimal configuration**** for the more challenging Humanoid task.
- (Optional) Pick the best configuration among single-Q, double-Q, and clipped double-Q, and run it on **Humanoid-v4** using **humanoid.yaml** (edit the config to use the best option). Plot the default configuration (in the previous section) and the best configuration together with environment steps on the *x*-axis and evaluation return on the *y*-axis.

Note: if you’d like to run training to completion (5M steps, 24h of training), you should get a proper, walking humanoid! You can run with videos enabled by using **-nvid 1**.

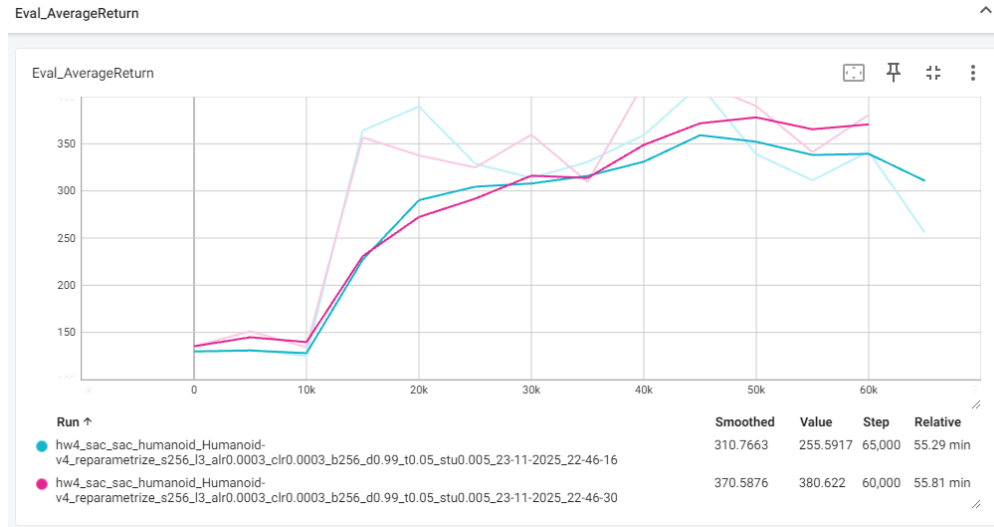


Figure 7: Final Comparison of Evaluation Return on Humanoid-v4 (Default vs. Best Configuration). This plot compares the performance of the Default (Single-Q, Cyan) and the Best (Double-Q, Pink) Q-value estimation strategies in the complex Humanoid environment.

1. Configuration Performance Comparison

This plot directly compares the performance of the two critical Q-value estimation strategies over the training duration.

- **Best Configuration (Double-Q, Pink):** Achieved a consistently higher and more stable return, with a

final Smoothed value of 370.5876 at 60K steps. The maximum value reached was 380.622.

- **Default Configuration (Single-Q, Cyan):** Demonstrated lower overall performance, with a final Smoothed value of 310.7663 at 65K steps.
- **Conclusion:** The Best Configuration (Double-Q) achieved an approximately 19% higher smoothed return than the Default Configuration (Single-Q) in this initial training phase.

2. Analysis of Strategy Effectiveness

The results strongly validate the selection of **Double Q-Learning** as the optimal configuration for the Humanoid-v4 task.

- **Single-Q Instability:** The Default Configuration (Single-Q) likely suffered from overestimation bias, which typically hinders learning stability in high-dimensional environments like Humanoid, resulting in lower sustained returns.
- **Double-Q Robustness:** The Best Configuration (Double-Q) successfully mitigates this overestimation by decoupling the target selection and evaluation networks. This approach provides a **more accurate and stable value estimate**, enabling the agent to learn the complex balance and locomotion required for the Humanoid environment more effectively, leading to the superior final return.
- **Learning Speed:** Both configurations show a rapid acceleration in performance after 10K steps, but the Double-Q agent maintains a higher ceiling throughout the training.

9 Submitting the code and experiment runs

Please submit the code, tensorboard logs, and the “report” in a single zip file, `hw4_[YourStudentID].zip`. Do not include videos as the file size should be less than 50MB. The structure of the submission file should be:

```
hw4_[YourStudentID].zip
├── hw4_[YourStudentID].pdf
├── data
│   ├── hw4_sac_...
│   │   └── events.out.tfevents.1567529456.e3a096ac8ff4
│   └── ...
├── gcb6206
│   ├── agents
│   │   └── sac_agent.py
│   └── ...
├── README.md
└── ...
```